# Trainer

https://huggingface.co/docs/transformers/main/en/main_classes/trainer#transf

## Detailed Documentation

### class transformers.Trainertransformers.Trainerhttps://g

class `transformers.Trainertransformers.Trainerhttps`://github.com/huggingface/t "model", "val": ": typing.Union[`transformers.modeling_utils.PreTrainedModel`, torch.nn.modules.module.Module, NoneType] = None"}, {"name": "args", "val": ": typing.Optional[`transformers.training_args.TrainingArguments`] = None"}, {"name": "data_collator", "val": ": typing.Optional[collections.abc.Callable[[list[typing.Any]], dict[str, typing.Any]]] = None"}, {"name": "train_dataset", "val": ": typing.Union[torch.utils.data.dataset.Dataset, torch.utils.data.dataset.IterableDataset, ForwardRef('datasets.Dataset'), NoneType] = None"}, {"name": "eval_dataset", "val": ": typing.Union[torch.utils.data.dataset.Dataset, dict[str, torch.utils.data.dataset.Dataset], ForwardRef('datasets.Dataset'), NoneType] = None"}, {"name": "processing_class", "val": ": typing.Union[`transformers.tokenization_utils_base.PreTrainedTokenizerBas`

`transformers.image_processing_utils.BaseImageProcessor`, `transformers.feature_extraction_utils.FeatureExtractionMixin`, `transformers.processing_utils.ProcessorMixin`, NoneType] = None"}, {"name": "model_init", "val": ": typing.Optional[collections.abc.Callable[..., `transformers.modeling_utils.PreTrainedModel`]] = None"}, {"name": "compute_loss_func", "val": ": typing.Optional[collections.abc.Callable] = None"}, {"name": "compute_metrics", "val": ": typing.Optional[collections.abc.Callable[[ `transformers.trainer_utils.EvalPred` dict]] = None"}, {"name": "callbacks", "val": ": typing.Optional[list[ `transformers.trainer_callback.TrainerCallback`]] = None"}, {"name": "optimizers", "val": ": tuple = (None, None)"}, {"name": "optimizer_cls_and_kwargs", "val": ": typing.Optional[tuple[type[torch.optim.optimizer.Optimizer], dict[str, typing.Any]]] = None"}, {"name": "preprocess_logits_for_metrics", "val": ": typing.Optional[collections.abc.Callable[[torch.Tensor, torch.Tensor], torch.Tensor]] = None"}]- **model** ([PreTrainedModel] (/docs/transformers/main/en/main_classes/model# `transformers.PreTrainedMod` or `torch.nn.Module`, *optional*) --

The model to train, evaluate or use for predictions. If not provided, a `model_init` must be passed.

[Trainer] (/docs/transformers/main/en/main_classes/trainer# `transformers.Trainer`) is optimized to work with the [PreTrainedModel] (/docs/transformers/main/en/main_classes/model# `transformers.PreTrainedMod` provided by the library. You can still use

your own models defined as `torch.nn.Module` as long as they work the same way as the Transformers

- **args** ([TrainingArguments] (/docs/transformers/main/en/main_classes/trainer# `transformers.TrainingA` *optional*) --

The arguments to tweak for training. Will default to a basic instance of [TrainingArguments] (/docs/transformers/main/en/main_classes/trainer# `transformers.TrainingArgum` with the

`output_dir` set to a directory named *tmp_trainer* in the current directory if not provided.

- **d

---

## add_callbacktransformers.Trainer.add_callbackhttps://github.cor

add_callbacktransformers.Trainer.add_callbackhttps://github.com/huggingface/tr "callback", "val": ""}]- **callback** ( `type` or [ `~transformers.TrainerCallback` ] ) --

A [TrainerCallback] (/docs/transformers/main/en/main_classes/callback# `transformers.TrainerCall` class or an instance of a [TrainerCallback] (/docs/transformers/main/en/main_classes/callback# `transformers.TrainerCall` In the

first case, will instantiate a member of that class.0

Add a callback to the current list of [TrainerCallback] (/docs/transformers/main/en/main_classes/callback# `transformers.TrainerCall`

---

## autocast_smart_context_managertransformers.Trainer.autocast_

autocast_smart_context_managertransformers.Trainer.autocast_smart_context_m "cache_enabled", "val": ": typing.Optional[bool] = True"}]

A helper wrapper that creates an appropriate context manager for `autocast` while feeding it the desired

arguments, depending on the situation. We rely on accelerate for autocast, hence we do nothing here.

---

## compute_losstransformers.Trainer.compute_losshttps://github.co

compute_losstransformers.Trainer.compute_losshttps://github.com/huggingface/ "model", "val": ": Module"}, {"name": "inputs", "val": ": dict"}, {"name": "return_outputs", "val": ": bool = False"}, {"name": "num_items_in_batch", "val": ": typing.Optional[torch.Tensor] = None"}]- **model** ( `nn.Module` ) --

The model to compute the loss for.

- **inputs** ( `dict[str, Union[torch.Tensor, Any]]` ) --

The input data for the model.

- **return_outputs** (`bool`, *optional*, defaults to `False`) --

Whether to return the model outputs along with the loss.

- **num_items_in_batch** (Optional[torch.Tensor], *optional*) --

The number of items in the batch. If num_items_in_batch is not passed,0The loss of the model along with its output if return_outputs was set to True

How the loss is computed by Trainer. By default, all models return the loss in the first element.

Subclass and override for custom behavior. If you are not using `num_items_in_batch` when computing your loss,

---

## compute_loss_context_managertransformers.Trainer.compute_lo

compute_loss_context_managertransformers.Trainer.compute_loss_context_man

A helper wrapper to group together context managers.

---

## create_model_cardtransformers.Trainer.create_model_cardhttps:

create_model_cardtransformers.Trainer.create_model_cardhttps://github.com/hu
"language", "val": ": typing.Optional[str] = None"}, {"name": "license", "val": ":
typing.Optional[str] = None"}, {"name": "tags", "val": ": typing.Union[str, list[str],
NoneType] = None"}, {"name": "model_name", "val": ": typing.Optional[str] =
None"}, {"name": "finetuned_from", "val": ": typing.Optional[str] = None"},

{"name": "tasks", "val": ": typing.Union[str, list[str], NoneType] = None"}, {"name": "dataset_tags", "val": ": typing.Union[str, list[str], NoneType] = None"}, {"name": "dataset", "val": ": typing.Union[str, list[str], NoneType] = None"}, {"name": "dataset_args", "val": ": typing.Union[str, list[str], NoneType] = None"}]- **language** ( `str` , *optional*) --

The language of the `model` (if applicable)

- **license** ( `str` , *optional*) --

The license of the model. Will default to the license of the pretrained model used, if the original

model given to the `Trainer` comes from a repo on the Hub.

- **tags** ( `str` or `list[str]` , *optional*) --

Some tags to be included in the metadata of the model card.

- **model_name** ( `str` , *optional*) --

The name of the model.

- **finetuned_from** ( `str` , *optional*) --

---

## create_optimizertransformers.Trainer.create_optimizerhttps://gith

create_optimizertransformers.Trainer.create_optimizerhttps://github.com/hugging

Setup the optimizer.

We provide a reasonable default that works well. If you want to use something else, you can pass a tuple in the

Trainer's init through `optimizers`, or subclass and override this method in a subclass.

---

## create_optimizer_and_schedulertransformers.Trainer.create_optii

create_optimizer_and_schedulertransformers.Trainer.create_optimizer_and_schec "num_training_steps", "val": ": int"}]

Setup the optimizer and the learning rate scheduler.

We provide a reasonable default that works well. If you want to use something else, you can pass a tuple in the

Trainer's init through `optimizers`, or subclass and override this `method` (or `create_optimizer` and/or

`create_scheduler`) in a subclass.

---

## create_schedulertransformers.Trainer.create_schedulerhttps://git

create_schedulertransformers.Trainer.create_schedulerhttps://github.com/huggin "num_training_steps", "val": ": int"}, {"name": "optimizer", "val": ": Optimizer = None"}]- **num_training_steps** (int) -- The number of training steps to do.0

Setup the scheduler. The optimizer of the trainer must have been set up either

before this method is called or

passed as an argument.

---

## evaluatetransformers.Trainer.evaluatehttps://github.com/huggingg

evaluatetransformers.Trainer.evaluatehttps://github.com/huggingface/transformer "eval_dataset", "val": ": typing.Union[torch.utils.data.dataset.Dataset, dict[str, torch.utils.data.dataset.Dataset], NoneType] = None"}, {"name": "ignore_keys", "val": ": typing.Optional[list[str]] = None"}, {"name": "metric_key_prefix", "val": ": str = 'eval'"}]- **eval_dataset** (Union[ `Dataset` , dict[str, `Dataset` ]], *optional*) --

Pass a dataset if you wish to override `self.eval_dataset` . If it is a [Dataset] (https://huggingface.co/docs/datasets/main/en/package_reference/main_classes# columns

not accepted by the `model.forward()` method are automatically removed. If it is a dictionary, it will

evaluate on each dataset, prepending the dictionary key to the metric name. Datasets must implement the

`__len__` method.

If you pass a dictionary with names of datasets as keys and datasets as values, evaluate will run

separate evaluations on each dataset. This can be useful to monitor how training

affects other

datasets or simply to get a more fine-grained evaluation.

When used with `load_best_model_at_end`, make sure `metric_for_best_model` references exactly one

of the datasets. If you, for example, pass in `{"data1": data1, "data2": data2}` for two datasets

---

## evaluation_looptransformers.Trainer.evaluation_loophttps://githu

evaluation_looptransformers.Trainer.evaluation_loophttps://github.com/huggingfa "dataloader", "val": ": DataLoader"}, {"name": "description", "val": ": str"}, {"name": "prediction_loss_only", "val": ": typing.Optional[bool] = None"}, {"name": "ignore_keys", "val": ": typing.Optional[list[str]] = None"}, {"name": "metric_key_prefix", "val": ": str = 'eval'"}]

Prediction/evaluation loop, shared by `Trainer.evaluate()` and `Trainer.predict()`.

Works both with or without labels.

---

## floating_point_opstransformers.Trainer.floating_point_opshttps://

floating_point_opstransformers.Trainer.floating_point_opshttps://github.com/hugg "inputs", "val": ": dict"}]- **inputs** (`dict[str, Union[torch.Tensor, Any]]`) --

The inputs and targets of the model.0 `int` The number of floating-point operations.

For models that inherit from [PreTrainedModel] (/docs/transformers/main/en/main_classes/model# `transformers.PreTrainedMod` uses that method to compute the number of floating point

operations for every backward + forward pass. If using another model, either implement such a method in the

model or subclass and override this method.

## get_batch_samplestransformers.Trainer.get_batch_sampleshttps

get_batch_samplestransformers.Trai